

UDFs Explained

DAX UDFs:
The Complete Guide



DEFINE 1

```
/// TargetLineBarChart creates an SVG bar chart with a target line. 2  
/// Returns: SVG string (data URI) for use in a table/matrix visual.
```

Update model: Overwrite function

```
FUNCTION UDF_TargetLineBarChart = 3
```

Function Parameters

```
(  
    actualMeasure      : SCALAR NUMERIC EXPR, 4  
    targetMeasure      : SCALAR NUMERIC EXPR,  
    dimensionColumn    : ANYREF EXPR,  
    chartHeight        : NUMERIC VAL = 35, 5  
    barHeight          : NUMERIC VAL = 28 ,  
    axisMinValue       : NUMERIC VAL = 0,  
    axisMaxValue       : NUMERIC VAL = 130,  
    belowColor         : STRING VAL = "#e74c3c",  
    aboveColor         : STRING VAL = "#2ecc71"  
    ) =>  
    ... 6
```

User-defined functions (UDFs) let you package a piece of DAX logic into a reusable function.

- 1 UDFs can be made in three tabs: DAX Query View, TMDL View and Model explorer
- 2 Comments between the function and the DEFINE statement show in IntelliSense
- 3 After FUNCTION, you define the function's name
- 4 For each parameter define a Type, Subtype, Parameter Mode
- 5 Set a default value with an equal sign
- 6 After ⇒ you write your DAX function

More details on
other pages →

User-defined functions (UDFs) let you package a piece of DAX logic (with its own parameters) into a reusable function, instead of repeating the same expression across many measures or columns. Once defined and saved to the model, a UDF works exactly like a native DAX function: you call it by name from a measure, calculated column, visual calculation, or even from inside another UDF.

1 Three Places To Build A UDF

-  • DAX Query View: write the DEFINE / FUNCTION syntax directly, test it instantly with EVALUATE, then click "Update model" to save it.
-  • TMDL View: use createOrReplace function syntax, then click Apply to push it to the model.
-  • Model Explorer: create a brand-new function via the formula bar, or browse/edit existing ones under the Functions node.

2 Comments That Show Up In IntelliSense

- Only triple-slash **///** comments placed between DEFINE and the FUNCTION line get picked up by IntelliSense as the function's description.
- Regular **//** or **/* */** comments are invisible to IntelliSense — easy mistake to make.

3 Naming The Function

- Right after the FUNCTION keyword, you assign the name that will show up everywhere you call it (e.g. UDF_TargetLineBarChart).
- Naming rules: letters, numbers, underscores, and periods only, no spaces or special characters.
- Periods are allowed for namespacing (e.g. Microsoft.PowerBI.MyFunc), but can't start/end a name or appear twice in a row.
- Can't reuse a reserved word or built-in DAX function name (measure, function, define, etc.) — and names must be unique in the model.

- Each parameter can optionally declare three things, in this order: [Type] [Subtype] [ParameterMode]. Skip all of them, and it defaults to **AnyVal val**.
- Type: what kind of input it accepts: AnyVal, Scalar, Table, or a reference type (AnyRef, ColumnRef, MeasureRef, TableRef, CalendarRef).
- Subtype (Scalar only): the exact data type: Numeric, Int64, Decimal, Double, String, DateTime, Boolean, Variant.
- **Parameter Mode**: when it's evaluated:
 - **VAL** = evaluated once, before the function runs, in the caller's context. Acts like a fixed variable — won't react to filter changes inside the function.
 - **EXPR** = evaluated inside the function body, every time it's referenced, in whatever context exists at that point. Required for anything filter-context-sensitive, like CALCULATE.
- Why it matters: a function that wraps CALCULATE to filter for "Red" products only works if its amount parameter is EXPR. With VAL, the value gets locked in before CALCULATE's filter is applied — so the filter silently does nothing, and you get the wrong number with no error.
- **▲** Reference types (AnyRef, ColumnRef, MeasureRef, TableRef, CalendarRef) must be EXPR — VAL isn't allowed for them.

Type	ParameterMode not specified	ParameterMode: val	ParameterMode: expr
(Not Specified) / AnyVal	val	val	expr
Scalar, Table	val	val	expr
AnyRef	expr	Not allowed	expr
CalendarRef, ColumnRef, MeasureRef, TableRef	expr	Not allowed	expr

- Add = and a value after a parameter to make it optional.
- Optional parameters can sit anywhere in the parameter list — but the function's minimum required argument count is still set by the right-most required parameter.
- The default expression can only reference things visible where the function is defined, not where it's later called — and it can't reference another optional parameter.

6

The Function Body

- Everything after $\Rightarrow$ is the function body — a normal DAX expression.
- It can be one line, use VAR/RETURN, or call other UDFs (functions can be nested).

Other Things To Know

- Up to 256 parameters per function, and No recursion
- UDFs can only be authored/edited in Power BI Desktop (not directly in the Service).
- Object-Level Security doesn't transfer to a function that references a secured measure or column.

Sources:

- Microsoft Learn:
<https://learn.microsoft.com/en-us/dax/best-practices/dax-user-defined-functions#useful-information-functions>
- SQLBI:
<https://www.sqlbi.com/articles/introducing-user-defined-functions-in-dax/>
- Tabular Editor:
<https://docs.tabulareditor.com/en/tutorials/udfs.html>



Thanks for Reading